阶段一

1.1

1. CPU (中央处理器)：负责执行所有复杂的计算指令，进行逻辑判断和任务调度，是计算机的运算和控制核心。 GPU (图形处理器)：拥有数千个流处理器核心，专为并行计算设计，能同时处理大量简单计算，主要用于3D图形渲染、视频编解码和深度学习加速。 主板：它是一块矩形电路板，提供各种插槽（如CPU、内存、PCIe插槽）和接口（如USB、网口），将所有硬件组件连接在一起，并让它们通过总线架构进行数据传输。

2. 电脑的“16G内存”：通常指的是运行内存，即RAM（Random Access Memory），也叫主存。它是CPU直接交换数据的内部存储器，用于临时存放正在运 行的程序和数据，读写速度极快，但断电后数据会丢失（易失性）。

   手机的“512G内存”：实际上指的是机身存储（ROM），即闪存（Flash Memory）。它的作用相当于电脑的硬盘，用于长期存放照片、视频、应用程序等数据，断电后数据不会丢失（非易失性）。

3. 电源（PSU）是电脑的心脏，为所有部件提供稳定、纯净的电力。选购时需要注意：

   额定功率：这是最重要的指标。要根据所有硬件的总功耗来选，并预留20%-30%的余量，比如整机峰值功耗600W，建议选择750W-850W的电源，以保证稳定运行和未来升级空间。

   认证与品质：优先选择通过80 PLUS认证（尤其是金牌及以上）的电源，这代表了更高的转换效率和更好的用料。

   接口与标准：确认电源提供足够的CPU供电接口（如双8Pin）和显卡供电接口。对于新显卡，建议选择支持ATX 3.0/3.1标准、带有原生12V-2x6接口 的电源。

4. 不一定。

   CPU性能：不能只看i3、i5、i7、i9的级别，代际和具体型号同样关键。

   代际碾压：一个几年前的旗舰i7，其性能可能完全比不过最新一代的i3。

   后缀区分：同一代中，带“K”的型号通常支持超频，性能更强；带“H”的是高性能移动版；带“U”的是低功耗移动版。

   怎么看：最直观的方法是查阅CPU性能天梯图，它能清晰地展示不同型号CPU的相对性能排名。

   GPU性能：同理，不能只看30系、40系。

   代际提升：新一代的显卡通常会有架构和能效上的巨大提升。例如，RTX 4070的性能甚至可以媲美上一代的旗舰RTX 3080。

   型号定位：在每一代中，60、70、80、90等数字代表了从主流到旗舰的性能定位。

5. “i7级CPU”：这是典型的文字游戏。它很可能是一个性能远不如真正i7的旧款或低端处理器。

   “16G高端内存”：内存的关键在于代数（DDR4/DDR5） 和频率，而不仅仅是容量。老板很可能用低频的DDR4内存冒充“高端”。

“性能比上一代高出一半的豪华GPU”：这是一个非常模糊且无法验证的说法。GPU性能需要看具体的型号（如RTX 4060），而不是这种笼统的描述。

“512G内存的手机”：这是最严重的概念错误。手机的512G是存储空间，不是运行内存。他故意混淆了存储和内存的概念。

“1.25T的超大硬盘”：这个容量本身没问题，但老板将其与“512G内存的手机”并列，进一步强化了其混淆“内存”与“存储”概念的意图。

“只要4000信用点”：综合以上模糊和误导性的描述，这套配置的实际价值很可能远低于4000信用点，老板是想利用信息差牟利。

1.2

1.

存储介质 应用部件 核心特点 作用 SRAM (静态随机存取存储器) CPU内部的高速缓存（Cache） 速度极快，功耗低，但集成度低、成本高、容量小 作为CPU和主存之间的高速缓冲，利 用程序局部性原理，极大地缓解了CPU与主存之间的速度差距。 DRAM (动态随机存取存储器) 计算机的主内存（内存条） 速度较快，集成度高、成本较低， 作为程序运行的主要工作区，为CPU提供快 但需要周期性刷新以保持数据，功耗较高 速的数据读写，是连接CPU和外部存储的桥梁

NAND Flash (闪存) 固态硬盘（SSD）、U盘、手机存储 非易失性（断电不丢失数据），读写速度快， 用于长期存储数据，其高速特性极大地提升了 体积小，功耗低 系统启动、程序加载和文件传输的速度。 HDD (机械硬盘) 大容量数据存储、服务器、数据中心 非易失性，容量极大，单位成本极低， 用于对速度要求不高但对容量和成本敏感的 但读写速度慢（有机械延迟） 冷数据存储，是海量数据备份的基石。

2. 寄存器：是CPU内部最快的存储单元，直接封装在CPU内核中，用于存放当前正在执行的指令和操作数。

   高速缓存：位于CPU内部或旁边（L1/L2/L3），用于存放CPU近期可能会用到的数据和指令，速度仅次于寄存器。

   只读存储器：是非易失性存储器，断电后数据不丢失，通常用来存储BIOS固件等启动程序。它位于主板上，是计算机启动时读取的第一段代码。

3. 物理结构：DRAM的每个存储单元由一个晶体管和一个电容器组成，一个单元存储1个比特（bit）。

   工作原理：它利用电容器中存储的电荷来表示数据。电容充电（有电荷）代表“1”，放电（无电荷）代表“0”。但由于晶体管存在微小的漏电流，电容中的电荷会逐渐流失，导致数据丢失，因此必须周期性地对电容进行刷新（充电或放电） 来维持数据。

4. Cache的根本原理是程序的局部性（Locality） 原理，具体包括： 时间局部性：如果一个数据或指令被访问，那么它在不久的将来很可能被再次访问（例如循环）。 空间局部性：如果一个存储单元被访问，那么它附近的存储单元很可能在不久的将来被访问（例如数组的顺序访问）。 Cache利用这两个特性，将最可能被访问的数据从慢速的主存中复制到快速且容量小的Cache中，从而大大提高了平均访问速度。

5. 将大矩阵分割成若干个小块（Tile），使得每个小块能够完整地装入Cache中。然后，程序在Cache内部对这些小块进行乘法运算，计算完后再进行下一 组小块的运算。通过调整循环顺序（例如，将k循环放到最内层），确保被反复使用的矩阵A和B的数据块能在Cache中停留更长时间。

运用原理：时间局部性和空间局部性。通过分块，一个被加载到Cache中的小块会在其被替换出去之前被多次重用（时间局部性），并且对小块内元素的 访问也是连续的，充分利用了空间局部性。这极大地减少了CPU访问慢速主存的次数，从而提升了程序性能。

阶段二

2.1

1. 虚拟地址：程序运行时使用的地址，是CPU在指令中生成的逻辑地址。每个进程都有自己独立的虚拟地址空间，仿佛独占整个内存。

   物理地址：实际内存条（DRAM）上的硬件地址，由内存控制器直接访问。

   引入虚拟内存的好处： 进程隔离与保护：每个进程有独立的虚拟地址空间，无法直接访问其他进程或内核的物理内存，提高了系统安全性和稳定性。 简化内存管理：程序不必关心物理内存的实际布局，只需使用连续的虚拟地址，操作系统负责映射到可能不连续的物理页框。 支持大于物理内存的程序：通过页面置换，可以把不常用的页换出到磁盘，使程序可以使用超过物理内存大小的地址空间。 共享内存与写时复制：多个进程可以共享同一物理页（如共享库），提高内存利用率。

2. 每个进程拥有独立的、私有的虚拟地址空间，大小固定为 4GB（32位系统，2^32 字节）。其中用户空间为 3GB，内核空间为 1GB。

   虚拟地址空间由页（Page） 组成，典型大小为 4KB（2^12 字节）。

   虚拟地址空间中的页通常划分为：代码段（.text）、数据段（.data）、BSS段（.bss）、堆（heap）、栈（stack）、共享库映射区、内核空间等。

   页（Page）：虚拟地址空间中的固定大小块，是虚拟内存管理的基本单位。 页框（Page Frame）：物理内存中的固定大小块，是物理内存管理的基本单位。 共同点：大小相同（通常为 4KB），页可以映射到任意页框，通过页表建立对应关系。

2.2

1. 页表是存储在内存中的一种数据结构，用于记录虚拟页到物理页框的映射关系。页表和页不是同一个东西：页是数据存储的单位，页表是管理映射关系的 元数据。

   MMU（内存管理单元）：CPU 内部硬件，负责将虚拟地址翻译为物理地址，并检查访问权限。它按照页表结构逐级查表。 TLB（转译后备缓冲区）：MMU 内部的小型高速缓存，存储最近使用过的页表项。地址翻译时先查 TLB，命中则直接得到物理地址，避免多次访问内存中的页表，大幅加速翻译。

   多级页表存储在物理内存中（由操作系统分配和管理）。 记录页表物理基地址的寄存器是 CR3（在 x86 架构中）。MMU 使用 CR3 找到页目录表的起始物理地址。

   二级页表将页表分为两级：页目录表（PDE）和页表（PTE）。 优势：节省内存空间。单级页表需要为整个虚拟地址空间（4GB）维护连续的大页表（4MB），而二级页表允许未使用的虚拟地址区域不分配页表，按需分配。 展现：进程实际使用的地址空间往往远小于 4GB，二级页表只为有效的页目录项分配下级页表，大大减少了内存占用。

部分 位数 含义 页目录索引 10 位（31~22） 索引页目录表，找到对应页表 页表索引 10 位（21~12） 索引页表，找到对应页表项 PTE 页内偏移 12 位（11~0） 页内偏移量，定位页内具体字节

2.

void translate_address(uint32_t virtual_address, bool is_write_access) { printf("----------\n"); printf("Translating Virtual Address: 0x%08X (%s access)\n", virtual_address, is_write_access ? "WRITE" : "READ");

```
// --- 步骤1: 从虚拟地址中提取索引和偏移 ---
uint32_t page_dir_index;
uint32_t page_table_index;
uint32_t offset;
// TO DO:
page_dir_index   = (virtual_address >> 22) & 0x3FF;  // 10位
page_table_index = (virtual_address >> 12) & 0x3FF;  // 10位
offset           = virtual_address & 0xFFF;          // 12位
// END OF TO DO

printf(" -> Page Dir Index: %u (0x%X)\n", page_dir_index, page_dir_index);
printf(" -> Page Table Index: %u (0x%X)\n", page_table_index, page_table_index);
printf(" -> Offset: %u (0x%X)\n", offset, offset);

// --- 步骤2: 查询页目录表 ---
// TO DO:
printf("  [*] Checking Page Directory Entry %u...\n", page_dir_index);
PDE pde = page_directory[page_dir_index];
if (!pde.present_bit) {
    printf("  [!] FAULT: Page Directory Entry not present. (Segmentation
Fault)\n");
    return;
}
printf("      -> PDE is present. Page table base address: %p\n",
(void*)pde.page_table_base);
// END OF TO DO

// --- 步骤3: 查询页表 ---
// TO DO:
PTE* page_table = pde.page_table_base;
PTE pte = page_table[page_table_index];
printf("  [*] Checking Page Table Entry %u...\n", page_table_index);
if (!pte.present_bit) {
    printf("  [!] FAULT: Page Table Entry not present. (Page Fault)\n");
    return;
}
printf("      -> PTE is present. Frame number: %u (0x%X)\n", pte.frame_number,
pte.frame_number);
// END OF TO DO

// --- 步骤4: 检查访问权限 ---
// TO DO:
printf("  [*] Checking access permissions...\n");
```

```
    if (is_write_access && !pte.rw_bit) {
        printf("  [!] FAULT: Write attempt on a read-only page. (Protection
    Fault)\n");
        return;
    }
    printf("      -> Access granted.\n");
    // END OF TO DO

    // --- 步骤5: 计算最终的物理地址 ---
    // TO DO:
    uint32_t physical_address = (pte.frame_number << 12) | offset;
    // END OF TO DO

    printf("  [SUCCESS] Translation complete.\n");
    printf("  Virtual Address 0x%08X  =>  Physical Address 0x%08X\n", virtual_address,
    physical_address);
```

}

2.3

1.

寄存器 作用 CR3 存放页目录表的物理基地址，MMU 据此开始地址翻译。 CR0 包含分页启用位（PG），控制是否开启分页机制。 CR2 发生缺页异常时，存放引发缺页的虚拟地址。 CR4 包含 PAE（物理地址扩展）等控制位，影响分页模式。 GDTR 全局描述符表寄存器，用于段式内存管理。 LDTR 局部描述符表寄存器。 段寄存器（CS、DS、SS、ES 等） 存放段选择子，与段描述符配合实现段式地址转换。

2.

从内存（DRAM）读取数据： CPU 发出虚拟地址。 MMU 查 TLB，若命中则得到物理地址；否则查页表，得到物理地址。 内存控制器根据物理地址访问 DRAM。 DRAM 将数据通过数据总线返回 CPU。

从存储（如 SSD/HDD）读取数据： CPU 发出读文件请求，文件系统根据路径找到 inode，得到数据块号。 若数据不在页缓存（Page Cache）中，则向磁盘驱动发出读请求。 磁盘驱动控制存储设备读取对应扇区/页到内存缓冲区。 数据从内核缓冲区复制到用户缓冲区，CPU 最终获得数据。

3.

缺页异常：当程序访问的虚拟页不在物理内存中（PTE 的 present 位为 0），MMU 触发的中断。 种类及处理：硬缺页（Major Page Fault）：页不在内存，且不在磁盘缓存中，需要从磁盘读取。OS 分配页框，从磁盘加载数据，更新页表，重新执行指令。 软缺页（Minor Page Fault）：页已在内存中（如被其他进程共享），但当前进程页表未映射。OS 只需建立映射，无需磁盘 I/O。 无效缺页：访问的地址非法（如空指针、越界）。OS 向进程发送 SIGSEGV 信号，通常导致段错误终止。

4.

页面替换：当物理内存不足，需要将某个物理页框换出到磁盘，腾出空间给新页使用时，选择哪个页被换出的过程。 进行方式：OS 维护页框的使用情况，当需要分配新页框时，根据替换算法选择一个牺牲页。若该页被修改过（脏页），需写回磁盘；否则直接丢弃。 常见替换算法：

LRU（最近最少使用）：替换最长时间未被访问的页。 Clock（时钟）算法：近似 LRU，使用访问位循环扫描。 FIFO：先进先出。 OPT（最优）：理论最优，实际无法实现。

64位系统与32位系统在虚拟内存和页表管理上的不同 对比项 32位系统 64位系统 虚拟地址空间大小 4GB 理论 2^64 字节，实际常用 48 位（256TB） 页表级数 二级页表 四级或五级页表（如 x86-64 四级） 页大小 通常 4KB 4KB，也支持 2MB、1GB 大页 地址划分 10+10+12 9+9+9+9+12（四级） CR3 指向页目录 指向 PML4 表 物理地址扩展 PAE 可支持 36 位物理地址 通常支持 48 位或更多物理地址 TLB 容量与结构 较小 更大、更复杂，支持多级 TLB

阶段三

3.1

1. 在 x86/x64 架构中，栈是从高地址向低地址方向增长的。 也就是说，每当有数据压入栈（push）时，栈顶指针（ESP/RSP）会减小；弹出数据（pop）时，栈顶指针会增大。
2. 栈帧（Stack Frame）是函数调用时在栈上开辟的一块内存区域，用于保存：函数的局部变量、函数参数（部分）、返回地址、 调用者的基址指针（EBP/RBP）、需要保存的寄存器
3. EBP/RBP（基址指针）：指向当前栈帧的底部（高地址端），在函数执行期间保持不变，用于稳定地访问局部变量和参数。 ESP/RSP（栈顶指针）：始终指向栈顶（低地址端），随着压栈和出栈动态变化。
4. 当执行 call 指令调用函数时，CPU 会自动将下一条指令的地址（即返回地址）压入栈中。 返回地址位于当前函数的 RBP 之上（高地址方向）8 字节处（64 位系统）。

3.2

1. gets() 函数不检查输入长度，会一直读取用户输入直到遇到换行符或 EOF。 如果用户输入的字符数超过 password[16] 的容量，多出的数据会覆盖栈上相邻的内存区域，包括保存的 RBP 和返回地址。 这种漏洞统称为 缓冲区溢出（Buffer Overflow） 或 栈溢出（Stack Overflow）。
2.

在 64 位系统中，假设无栈对齐填充，栈布局如下（从低地址到高地址）：

区域 大小 password[16] 16 字节 保存的旧 RBP 8 字节 返回地址 8 字节

要覆盖返回地址为 0x401156，输入字符串结构应为：

16 字节：填充 password 数组（任意字符，如 'A'） 8 字节：覆盖保存的旧 RBP（任意字符，如 'B'） 8 字节：写入目标地址 0x401156 的小端序表示

目标地址 0x0000000000401156 的小端序字节序列为：56 11 40 00 00 00 00 00

因此，输入字符串的十六进制形式为：

41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 （16 个 'A'） 42 42 42 42 42 42 42 42 （8 个 'B'） 56 11 40 00 00 00 00 00 （返回地址）

总长度为 32 字节。当 gets 读取该字符串时，会依次覆盖 password、旧 RBP，最终将返回地址改写为 0x401156。函数 verify_identity 执行 ret 时，就会跳转到 grant_access。

3.3

Stack Canary（栈金丝雀）

原理 在函数栈帧中，编译器会在局部变量与返回地址之间插入一个随机值，称为 canary（金丝雀）。函数返回前，会检查这个 canary 是否被修改：

正常情况：canary 保持原值，函数正常返回。

发生栈溢出：攻击者用输入数据覆盖缓冲区时，为了覆盖返回地址，必然会先覆盖 canary。函数返回前检查发现 canary 被篡改，立即调用 __stack_chk_fail 终止程序，攻击者无法劫持控制流。

防御效果 攻击者无法在不触碰 canary 的情况下覆盖返回地址。

随机 canary 每次程序运行都不同，难以预测。

有效阻止了经典的栈溢出劫持返回地址的攻击。

局限性 不防堆溢出、格式化字符串等其他漏洞。

如果攻击者能泄露 canary 的值（如通过信息泄露漏洞），仍可绕过。

对某些不覆盖返回地址的攻击无效。